

Automatic Rigidity for Collatz-Invariant 15-Automatic Sequences

Muteki

Independent Researcher (Japan)

GitHub: <https://github.com/MutekiResearch/ARC-Collatz>

Lean 4 formally verified version 1.0 (the ARC15 main proof does not use Cobham's theorem)

September 22, 2026

Abstract

Let $a = (a_n)_{n \geq 0}$ be a finite-valued sequence. We prove that if a is 15-automatic and satisfies

$$a_{2n} = a_n, \quad a_{2n+1} = a_{3n+2} \quad (n \geq 0),$$

then a is constant on the positive integers. The value a_0 remains genuinely free. Equivalently, every finite-valued 15-automatic coloring invariant under the shortcut Collatz map is trivial on $\mathbb{N}_{>0}$.

Base 15 is a useful odd-composite stress test. The conclusion is not obtained by combining the previous base-3 and base-5 results: 15-automaticity does not in general imply either 3-automaticity or 5-automaticity. Instead, the proof develops the doubled-loop dyadic pumping route directly in base 15. For a pump loop of length L , the doubled block produces

$$Q = 15^{2L}, \quad Q \equiv 1 \pmod{8}$$

(in fact $Q \equiv 1 \pmod{32}$). Writing

$$G_k = 1 + Q + \cdots + Q^{k-1},$$

the formal arithmetic proves the exact valuation identity

$$\nu_2(G_j - G_k) = \nu_2(j - k) \quad (j > k).$$

Consequently, the first 2^r values G_k permute all residues modulo 2^r . After restoring an arbitrary positive pump increment $d = 2^s(2v+1)$, the pumped numerical family fills every deeper dyadic child of its parent cylinder.

The automaton side identifies doubled-pumped base-15 words exactly with an affine family $N_0 + dG_k$, proves $d > 0$, and supplies the exact two-adic factorization above. Hence an infinite 15-automatic Boolean support cannot be dyadically nowhere thick. Applying this topological pumping theorem to the adjacent-change set $D_a = \{n : a_n \neq a_{n+1}\}$ makes D_a finite: shortcut invariance makes D_a dyadically nowhere thick, while 15-automaticity makes its Boolean characteristic sequence 15-automatic, including the carry case at the end of a base-15 residue block. Thus a is eventually constant, and $a_{2n} = a_n$ propagates the tail value to every positive integer.

The complete chain was formalized in Lean 4 / Mathlib, with final endpoint `arc15_final`, standard automaticity audit `arc15_kernel_iff_zero_digit_standard_msd`, and sharpness theorem `arc15_zero_value_is_genuinely_free`. Successful project builds report no `sorryAx`; the audited endpoints depend only on standard Lean/Mathlib principles such as `propxt`, `Classical.choice`, and `Quot.sound`.

Keywords: Collatz conjecture; automatic sequence; 15-kernel; finite automata; odd composite base; pumping; 2-adic valuation; dyadic residue filling; Lean 4.

2020 Mathematics Subject Classification: Primary 11B85; Secondary 11B83, 68Q45.

1 Introduction

Define the shortcut Collatz map $T : \mathbb{N}_0 \rightarrow \mathbb{N}_0$ by

$$T(2n) = n, \quad T(2n+1) = 3n+2 \quad (n \geq 0). \quad (1)$$

For background on the $3x+1$ problem, see Lagarias [5]. A sequence $a : \mathbb{N}_0 \rightarrow \Delta$ is T -invariant when $a_n = a_{T(n)}$ for every n . Equivalently,

$$a_{2n} = a_n, \quad a_{2n+1} = a_{3n+2} \quad (n \geq 0). \quad (2)$$

The present project asks how finite-state regularity interacts with invariance along every edge of the shortcut-Collatz functional graph. This is distinct from asking an automaton to compute a Collatz iterate, to recognize bounded-depth predecessor sets, or to describe a base-conversion transducer; those neighboring themes have been studied in several forms [6, 7, 8, 3, 2].

Theorem 1.1 (Automatic rigidity in base 15; ARC15). *Let Δ be a finite alphabet and let $a = (a_n)_{n \geq 0} \in \Delta^{\mathbb{N}_0}$ be a 15-automatic sequence. Assume (2) for every $n \geq 0$. Then there exists $c \in \Delta$ such that*

$$a_n = c \quad (n \geq 1).$$

Theorem 1.1 does not assume the Collatz conjecture. It is a rigidity theorem for one particular class of invariant colorings. The proof combines a base-independent local synchronization statement coming from shortcut invariance with a base-15 doubled-loop pumping theorem for automatic Boolean supports.

2 Why base 15 is a genuine new stress test

Base 15 is composite, with $15 = 3 \cdot 5$. Nevertheless, Theorem 1.1 does not follow formally from the previously established ARC3 and ARC5 statements. A sequence can be 15-automatic without being 3-automatic or 5-automatic; automaticity in a product base does not generally descend to either factor without an additional structural argument.

Thus ARC15 tests whether the local/dyadic mechanism survives beyond prime bases. The answer is yes, and the key numerical object is a doubled pump loop. Since $15 \equiv -1 \pmod{16}$, a single loop factor 15^L alternates with the parity of L , whereas

$$Q := 15^{2L} \equiv 1 \pmod{32}, \quad (3)$$

so in particular $Q \equiv 1 \pmod{8}$.

3 Geometric pumping sums and exact two-adic valuation

For a positive loop length L , define

$$Q = 15^{2L}, \quad G_0 = 0, \quad G_k = 1 + Q + \cdots + Q^{k-1} \quad (k \geq 1).$$

The elementary splitting identity is

$$G_{k+d} = G_k + Q^k G_d. \quad (4)$$

The central arithmetic theorem is stronger than a parity statement.

Lemma 3.1 (Valuation rigidity). *For every $d > 0$,*

$$\nu_2(G_d) = \nu_2(d).$$

Consequently, for every $j > k$,

$$\nu_2(G_j - G_k) = \nu_2(j - k).$$

Proof sketch. Odd d gives an odd geometric sum and hence both valuations are zero. For even d , the formal proof combines the factorization

$$(Q - 1)G_d = Q^d - 1$$

with the two-adic lifting-the-exponent identity for $Q^d - 1$. Because $Q \equiv 1 \pmod{4}$, one has $\nu_2(Q + 1) = 1$, and cancellation of the $Q - 1$ contribution leaves exactly $\nu_2(d)$. Equation (4) then gives the shifted difference form because Q^k is odd.

The Lean endpoint is `arc15_padicValNat_G_sub_eq_of_lt`. This is the arithmetic engine behind every later residue-filling statement.

4 Dyadic residue permutation and child filling

Fix $r \geq 0$ and consider

$$k \mapsto G_k \pmod{2^r}, \quad 0 \leq k < 2^r.$$

Proposition 4.1 (Dyadic residue permutation). *The first 2^r values $G_0, \dots, G_{2^r-1}$ represent every residue modulo 2^r exactly once.*

Proof sketch. If $G_j \equiv G_k \pmod{2^r}$ with $0 \leq k < j < 2^r$, then 2^r divides $G_j - G_k$. Lemma 3.1 implies $r \leq \nu_2(j - k)$, so $2^r \mid (j - k)$. But $0 < j - k < 2^r$, a contradiction. Thus the residue map is injective, hence bijective on the finite set of 2^r residues. The formal theorem is `arc15ResidueMap_bijective`.

Now let a numerical pump increment be written uniquely in the form

$$d = 2^s(2v + 1), \quad d > 0. \tag{5}$$

Multiplication by the odd factor $2v + 1$ permutes residues modulo every power of two. Therefore an affine pumped family

$$N_k = N_0 + dG_k \tag{6}$$

visits every residue class modulo 2^{s+r} lying over its parent class modulo 2^s . This is the dyadic child-filling statement formalized in `arc15_pumpedFamily_hits_every_child_of_exact_two_factor`.

Definition 4.2 (Dyadically nowhere thick). A set $S \subseteq \mathbb{N}_0$ is dyadically nowhere thick if, inside every dyadic parent residue class, there exists a deeper positive dyadic child residue class contained in that parent and disjoint from S .

A support that contains all values of a family (6) with the exact factorization (5) cannot satisfy Definition 4.2; the pumped family fills the child that was supposed to be empty. This collision is packaged by `arc15_stage5A_nowhereThick_collision`.

5 From automaton loops to the exact numerical family

The preceding arithmetic becomes relevant because a long accepted base-15 word contains a repeated state. If $x = abc$ is a pumping decomposition and reading b returns the automaton to the same state, then reading the doubled block bb also returns to that state. Hence the words

$$a(bb)^k c \quad (k \geq 0)$$

remain accepted.

The formal development does not stop at an abstract pumping lemma. It computes the exact natural number represented by every pumped word. With $L = |b|$,

$$\text{value}_{15}(a(bb)^k c) = N_0 + dG_k,$$

where G_k is built from $Q = 15^{2L}$. This equality is the theorem `arc15_msdValue_doublePumpedWord_eq_pumpedFamily`. The increment d is then proved strictly positive for canonical positive words and factored as in (5). These handoffs are formalized by `arc15_stage5D_positive_numeric_pumping_handoff` and `arc15_stage5E_exact_dyadic_pumping_handoff`.

6 Topological pumping for 15-automatic supports

Let $s : \mathbb{N}_0 \rightarrow \{0, 1\}$ be generated by a finite base-15 MSD-DFAO, with support $S = \text{supp}(s)$. If S is infinite, it contains arbitrarily large integers; therefore it contains canonical support words whose lengths exceed the number of automaton states. Such a word admits the doubled-loop decomposition above. The resulting numerical family stays inside S and fills every deeper dyadic child of its parent cylinder, contradicting dyadic nowhere-thickness.

Theorem 6.1 (Base-15 topological pumping). *If a Boolean 15-automatic sequence has dyadically nowhere-thick support, then its support is finite.*

The explicit MSD-DFAO form is `arc15_stage5F_msd_topological_pumping`; the finite-kernel form is `arc15_stage5G_kernel_topological_pumping`. No use of Cobham's theorem occurs in this route.

7 The adjacent-change set

Define

$$D_a = \{n \geq 0 : a_n \neq a_{n+1}\}.$$

Let χ_a be its Boolean characteristic sequence. We need two facts.

First, finite 15-kernel of a implies finite 15-kernel of χ_a . A kernel residual compares

$$a_{15^e n + r} \quad \text{and} \quad a_{15^e n + r + 1}.$$

If $r + 1 < 15^e$, both terms are ordinary kernel states. At the boundary $r + 1 = 15^e$, the second term is

$$a_{15^e(n+1)},$$

the one-step successor of the zero-residue kernel state. A finite kernel therefore generates only finitely many disagreement comparisons. This is `arc15_stage5H_change_automaticity`.

Second, shortcut invariance itself makes D_a dyadically nowhere thick. The base-independent local synchronization theorem gives, inside every prescribed dyadic parent class, a deeper positive child class on which

$$a_n = a_{n+1}$$

for all n in that child. Hence the child is disjoint from D_a .

Combining these facts with Theorem 6.1 gives

Theorem 7.1 (Finite change set). *If a is 15-automatic and shortcut-Collatz invariant, then D_a is finite.*

The corresponding formal handoff is `arc15_stage5I_change_set_finite_handoff`.

8 Eventual constancy and positive rigidity

Because D_a is finite, choose N strictly larger than every element of D_a . Then $a_n = a_{n+1}$ for all $n \geq N$, so induction gives a value c such that

$$a_n = c \quad (n \geq N).$$

This is `arc15_stage5J_eventual_constancy`.

Shortcut invariance on the even branch gives $a_{2n} = a_n$, hence

$$a_{2^k n} = a_n \quad (k, n \geq 0).$$

For any $n > 0$, choose k so that $2^k n \geq N$. Then

$$a_n = a_{2^k n} = c.$$

Thus Theorem 1.1 follows. Lean packages this as `arc15_stage5K_positive_rigidity` and finally `arc15_final`.

9 Sharpness at zero

The exclusion of zero is exact.

Proposition 9.1 (The zero value is genuinely free). *For arbitrary values x, y , define*

$$b_0 = x, \quad b_n = y \quad (n \geq 1).$$

Then b is 15-automatic and satisfies the shortcut-Collatz branch identities for every $n \geq 0$.

The base-15 kernel of b contains at most two sequences: b itself and the constant sequence y . The formal theorem `arc15_zero_value_is_genuinely_free` proves automaticity in the usual canonical MSD convention and both branch identities. Thus no theorem with the same hypotheses can force a_0 to equal the common positive value.

For Boolean sequences, Theorem 1.1 says that every 15-automatic subset of $\mathbb{N}_0$ invariant under shortcut-Collatz edges is either empty or all of $\mathbb{N}_{>0}$ on the positive domain; membership of zero remains independent.

10 Relation to previous ARC cases and generalization outlook

ARC15 provides a different kind of evidence from the prime-base cases. The table summarizes the main proof routes.

	ARC5	ARC7	ARC15
Base type	odd prime	odd prime	odd composite $3 \cdot 5$
Pumping factor	5^L	7^{2L}	15^{2L}
Key congruence	$5^L \equiv 1 \pmod{4}$	$7^{2L} \equiv 1 \pmod{8}$	$15^{2L} \equiv 1 \pmod{32}$
Dyadic engine	pumping + thinness	doubled-loop filling	doubled-loop valuation + filling
Cobham in main route	no	no	no
Conclusion	positive constancy	positive constancy	positive constancy

The doubled-loop mechanism strongly suggests an odd-base generalization, because every odd integer b satisfies $b^2 \equiv 1 \pmod{8}$. ARC15 is not, by itself, that general theorem: the present Lean files are explicitly base-15 and the automaton/numeric interfaces are instantiated at 15. A separate general-base formalization is therefore required.

Targeted literature searches during the project did not identify a theorem stated in the same form as Theorem 1.1. This is a provisional literature conclusion only; an equivalent consequence under different terminology may exist. The paper makes no exhaustive priority claim.

11 Lean 4 verification

The final finite-kernel theorem is

`arc15_final,`

and the paper-facing usual canonical MSD interface is

`arc15_main_zero_digit_standard_msd_form.`

The project was compiled stage by stage and then closed by a full `lake build`; the completed build reported 8884 jobs after inclusion of the sharpness file.

Formal theorem	Main verified content
<code>arc15_padicValNat_G_sub_eq_of_lt</code> <code>arc15ResidueMap_bijective</code>	Exact two-adic valuation of geometric-sum differences. The first 2^r geometric sums permute all residues modulo 2^r .
<code>arc15_pumpedFamily_hits_every_child_of_exact_two_factor</code> <code>arc15_stage5A_nowhereThick_collision</code>	Exact dyadic child filling for $N_0 + 2^s(2v+1)G_k$.
<code>arc15_stage5B_double_loop_pumping_handoff</code> <code>arc15_stage5C_base15_numeric_pumping_handoff</code>	A support-preserving child-filling family contradicts dyadic nowhere-thickness. Doubled automaton loop can be pumped arbitrarily. Pumped base-15 word values equal the affine geometric family.
<code>arc15_stage5D_positive_numeric_pumping_handoff</code> <code>arc15_stage5E_exact_dyadic_pumping_handoff</code>	Pump increment is strictly positive on positive canonical words. Positive increment factors exactly as $2^s(2v+1)$ and stays in support.
<code>arc15_stage5F_msd_topological_pumping</code> <code>arc15_stage5G_kernel_topological_pumping</code> <code>arc15_stage5H_change_automaticity</code>	MSD-DFAO topological pumping theorem. Finite-kernel version of topological pumping. Adjacent-change indicator is 15-automatic; carry handled explicitly.
<code>arc15_stage5I_change_set_finite_handoff</code>	Change set is automatic, dyadically nowhere thick, and finite.
<code>arc15_stage5J_eventual_constancy</code> <code>arc15_stage5K_positive_rigidity</code>	Finite changes imply an eventually constant tail. Doubling invariance propagates the tail value to all positive integers.
<code>arc15_final</code> <code>arc15_kernel_iff_zero_digit_standard_msd</code>	Packaged ARC15 rigidity theorem. Equivalence with the usual canonical MSD automaticity interface.
<code>arc15_zero_value_is_genuinely_free</code>	Sharpness: a_0 can be chosen independently.

Axiom audit

The major endpoints contain explicit `#print axioms` checks. The final theorem, automaticity audit, and sharpness theorem report only standard dependencies `propext`, `Classical.choice`, and `Quot.sound` (some elementary intermediate theorems use a subset). No `sorryAx` appeared in the successful final build reports. Formal verification certifies the encoded proof chain; it does not replace independent review of the mathematical modeling choices or literature context.

Disclosure of generative-AI assistance

Generative AI (OpenAI ChatGPT, GPT-5.6 Sol) was used for mathematical exploration, theorem organization, Lean 4 proof-code assistance, compile-error diagnosis, adversarial checking, literature-search assistance, manuscript drafting, and repository organization. The AI system is not an author or a bibliographic source. Responsibility for the released material remains with the human author.

References

References

- [1] J.-P. Allouche and J. Shallit, *Automatic Sequences: Theory, Applications, Generalizations*, Cambridge University Press, Cambridge, 2003.
- [2] B. Boigelot, On iterating linear transformations over recognizable sets of integers, *Theoret. Comput. Sci.* 309 (2003), 413–468. doi:10.1016/S0304-3975(03)00314-1.
- [3] D. Caucal and C. Rispal, On the powers of the Collatz function, in *Machines, Computations, and Universality*, Lecture Notes in Computer Science, Springer, 2025, 115–129. doi:10.1007/978-3-031-81202-6_8.
- [4] A. Cobham, On the base-dependence of sets of numbers recognizable by finite automata, *Math. Systems Theory* 3 (1969), 186–192. doi:10.1007/BF01746527.
- [5] J. C. Lagarias, The $3x + 1$ problem: an overview, in J. C. Lagarias (ed.), *The Ultimate Challenge: The $3x + 1$ Problem*, American Mathematical Society, Providence, RI, 2010, 3–29; arXiv:2111.02635, 2021.
- [6] J. Shallit and D. A. Wilson, The “ $3x + 1$ ” problem and finite automata, *Bull. European Assoc. Theor. Comput. Sci.* 46 (1992), 182–185.
- [7] T. Sterin, Binary expression of ancestors in the Collatz graph, in S. Schmitz and I. Potapov (eds.), *Reachability Problems*, Lecture Notes in Computer Science 12448, Springer, 2020, 115–130. doi:10.1007/978-3-030-61739-4_8.
- [8] T. Sterin and D. Woods, The Collatz process embeds a base conversion algorithm, in S. Schmitz and I. Potapov (eds.), *Reachability Problems*, Lecture Notes in Computer Science 12448, Springer, 2020, 131–147. doi:10.1007/978-3-030-61739-4_9.